GURU NANAK KHALSA COLLEGE

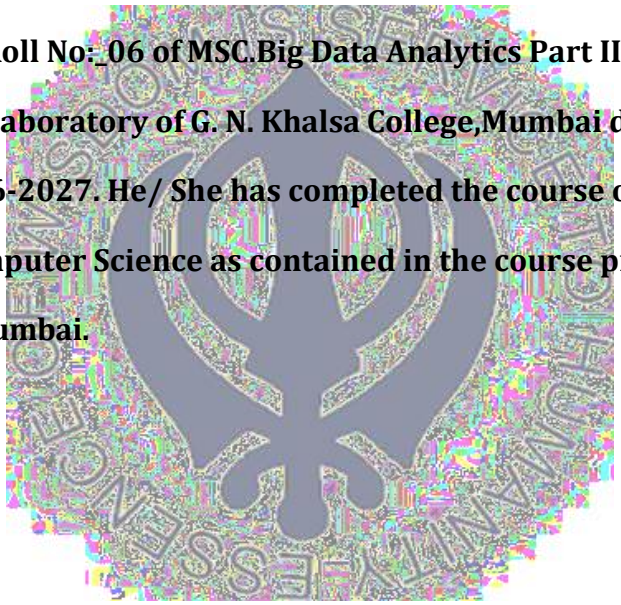
AUTONOMOUS (UNIVERSITY OF MUMBAI)

MUMBAI-400 019

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

CERTIFIED that the practical's, assignments duly signed, were performed by Mr/Ms Ritik Rahi. Roll No:_06 of MSC.Big Data Analytics Part II for the Computer Science Laboratory of G. N. Khalsa College,Mumbai during the academic year 2026-2027. He/ She has completed the course of Laboratory assignments in Computer Science as contained in the course prescribed by the University of Mumbai.



Sign of Student

Date _____

Professor-in-charge

1] _____

Date _____

2] _____

Date _____

Head Dept. of Computer Science

Date _____

Sign of Examiner's

1] _____

Date _____

2] _____

Date _____

INDEX

Sr. No.	Name of Practical	Page No.	Date	Sign
1	Study of Basic Linux Commands	3		
2	Shell Programming	5		
3	Process Creation using fork() – Parent, Child, Orphan and Zombie Processes	14		
4	CPU Scheduling Algorithms – FCFS, SJF, Priority and Round Robin	18		
5	Banker's Algorithm for Deadlock Avoidance	24		
6	Parallel Thread Management using POSIX Threads (pthread)	27		
7	Dynamic Memory Allocation – First Fit, Best Fit and Worst Fit	29		
8	Page Replacement Algorithms – FIFO, LRU and Optimal	32		
9	Type-I and Type-II Hypervisors and VM Setup using a Type-II Hypervisor	35		
10	Operating System Virtualization – Ubuntu VM using VMware Workstation	37		

Practical No. 1: Study of Basic Linux Commands

Aim: To study and execute basic Linux commands for file management, directory operations, file permissions, system information, and process management.

- ✓ **Theory/Description:**

Linux is an open-source, Unix-like operating system made up of the **kernel**, which manages hardware, processes and memory, and the **shell**, which reads the commands typed in the terminal and passes them to the kernel. Ubuntu uses the **bash** shell. Every command has the form `command [options] [arguments]`, and help for any command is available through `man <command>`.

All files are arranged in a single tree starting at the root directory `/`. A path starting from `/` is an absolute path, while a path starting from the current directory is a relative path. Every file has read (r = 4), write (w = 2) and execute (x = 1) permissions for the owner, group and others; for example `chmod 755` gives rwx to the owner and r-x to everyone else.

Commands Used

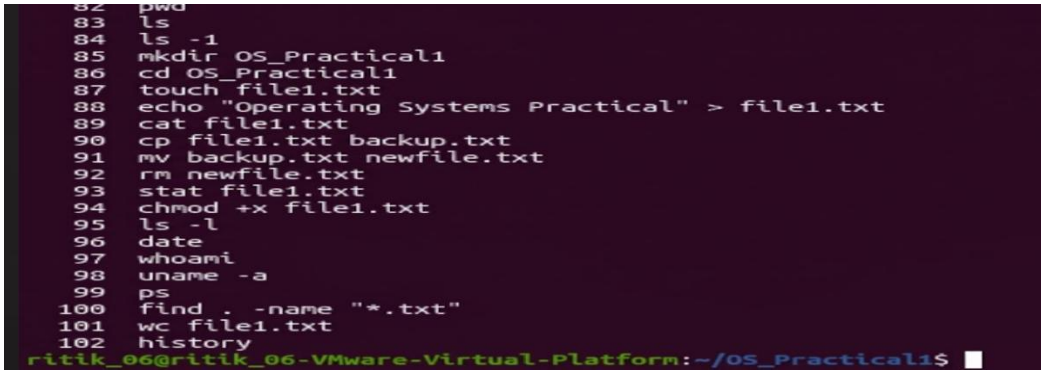
Sr. No.	Command	Purpose	Example
1	pwd	Display current directory	pwd
2	ls	List files and directories	ls -l
3	cd	Change directory	cd OS_Practical
4	mkdir	Create directory	mkdir OS_Practical
5	touch	Create file	touch file1.txt
6	echo	Display/write text into a file	echo "Operating Systems Practical" > file1.txt
7	cat	Display file contents	cat file1.txt
8	cp	Copy files	cp file1.txt backup.txt
9	mv	Move/Rename files	mv backup.txt newfile.txt
10	rm	Delete files	rm newfile.txt
11	stat	Display file information	stat file1.txt
12	chmod	Change file permissions	chmod +x file1.txt
13	date	Display current date and time	date
14	whoami	Show current user	whoami
15	uname	Display system information	uname -a
16	ps	Display running processes	ps
17	top	Monitor running processes	top
18	find	Search files	find . -name "*.txt"
19	wc	Count lines, words, and characters	wc file1.txt
20	history	Display command history	history

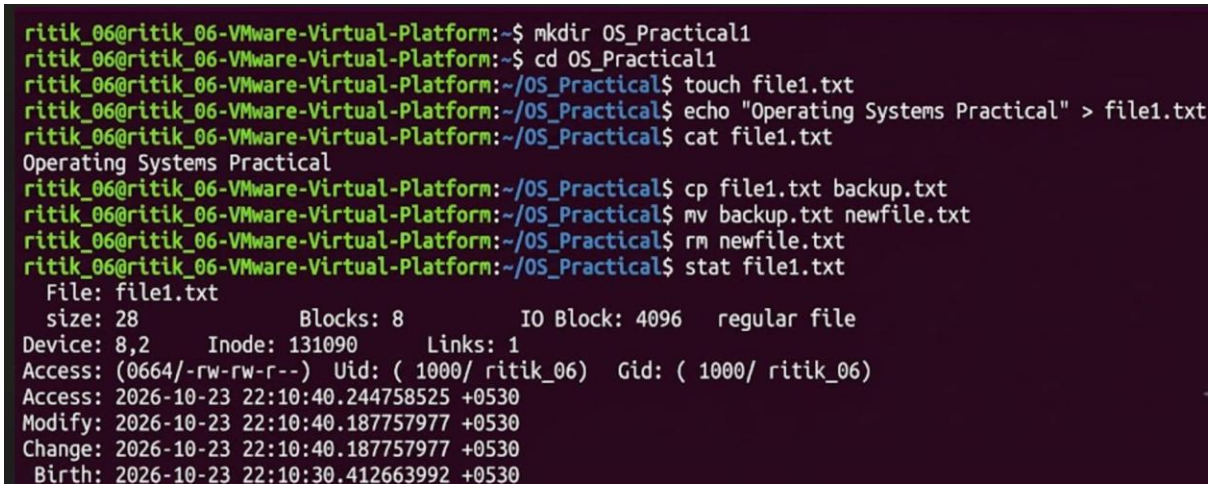
Procedure

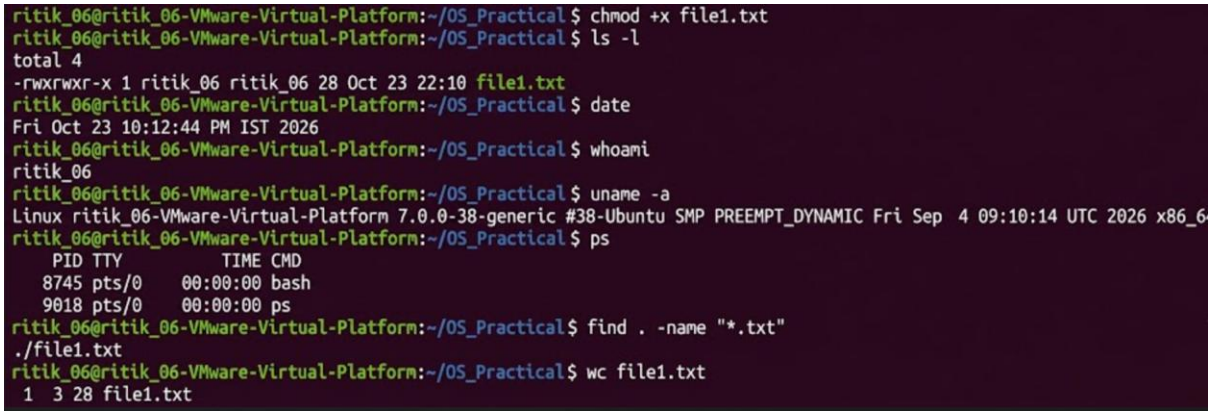
1. Open the Terminal (Ctrl + Alt + T).
2. Display the current working directory: `pwd`

3. List the files and directories: `ls` and `ls -l`
4. Create a new directory: `mkdir OS_Practical`
5. Move to the newly created directory: `cd OS_Practical`
6. Create an empty file: `touch file1.txt`
7. Write text into the file: `echo "Operating Systems Practical" > file1.txt`
8. Display the contents of the file: `cat file1.txt`
9. Create a copy of the file: `cp file1.txt backup.txt`
10. Rename the copied file: `mv backup.txt newfile.txt`
11. Delete the renamed file: `rm newfile.txt`
12. Display detailed information of the file: `stat file1.txt`
13. Change the file permission: `chmod +x file1.txt` and verify using `ls -l`
14. Display the current date and time: `date`
15. Display the current logged-in user: `whoami`
16. Display system information: `uname -a`
17. Display running processes: `ps`, and monitor them using `top` (press q to exit)
18. Search for all text files: `find . -name "*.txt"`
19. Count the lines, words and characters: `wc file1.txt`
20. Display the command history: `history`

Result: Successfully studied and executed basic Linux commands for file management, directory operations, file permissions, system information, and process management.






ritik_06@ritik_06-VMware-Virtual-Platform:~/OS_Practical$ chmod +x file1.txt
ritik_06@ritik_06-VMware-Virtual-Platform:~/OS_Practical$ ls -l
total 4
-rwxrwxr-x 1 ritik_06 ritik_06 28 Oct 23 22:10 file1.txt
ritik_06@ritik_06-VMware-Virtual-Platform:~/OS_Practical$ date
Fri Oct 23 10:12:44 PM IST 2026
ritik_06@ritik_06-VMware-Virtual-Platform:~/OS_Practical$ whoami
ritik_06
ritik_06@ritik_06-VMware-Virtual-Platform:~/OS_Practical$ uname -a
Linux ritik_06-VMware-Virtual-Platform 7.0.0-38-generic #38-Ubuntu SMP PREEMPT_DYNAMIC Fri Sep 4 09:10:14 UTC 2026 x86_6
ritik_06@ritik_06-VMware-Virtual-Platform:~/OS_Practical$ ps
PID TTY TIME CMD
8745 pts/0 00:00:00 bash
9018 pts/0 00:00:00 ps
ritik_06@ritik_06-VMware-Virtual-Platform:~/OS_Practical$ find . -name "*.txt"
./file1.txt
ritik_06@ritik_06-VMware-Virtual-Platform:~/OS_Practical$ wc file1.txt
1 3 28 file1.txt

Practical No. 2: Shell Programming

Aim: To write and execute shell scripts using input/output statements, decision-making, looping, and multi-level branching in Linux.

✓ **Theory/Description:**

A **shell script** is a text file containing a sequence of commands that the shell executes one after another, and is used to automate repetitive tasks. The first line, called the **shebang** (`#!/bin/bash` or `#!/bin/sh`), tells the system which shell should run the script. The script is made executable with `chmod +x` and run with `./script.sh`.

Variables are assigned without spaces (`name="PJ"`) and read using `$name`. Input is taken with `read` and output displayed with `echo`, while arithmetic is done using `$(( ))`. Conditions are written inside `[ ]` using the operators below, and the program flow is controlled using `if-elif-else`, `case`, `for` and `while`. Features such as `{1..5}` and `for ((i=1; i<=5; i++))` work only in bash, not in sh.

Operator	Meaning	Operator	Meaning
-eq	Equal to	-ne	Not equal to
-gt	Greater than	-lt	Less than
-ge	Greater than or equal to	-le	Less than or equal to

Commands Used

Sr. No.	Command	Purpose	Example
1	nano	Create/edit a shell script	nano script.sh
2	echo	Display output	echo "Hello"
3	read	Accept user input	read name
4	if	Execute statements based on a condition	if [$a -gt $b]
5	elif	Check multiple conditions	elif [$a -eq $b]
6	else	Execute alternative statements	else
7	case	Multi-way branching	case $choice in
8	for	Execute statements repeatedly	for i in 1 2 3
9	while	Repeat while a condition is true	while [$i -le 5]
10	chmod	Make the script executable	chmod +x script.sh
11	./script.sh	Execute the shell script	./script.sh

Procedure

1. Open the Terminal and create a script: `nano script.sh`
2. Write the program, save with Ctrl + S and exit with Ctrl + X.
3. Give execute permission: `chmod +x script.sh`
4. Execute the script: `./script.sh`
5. Repeat the steps for each of the following programs.

A) Simple Shell Program

Program:

```
#!/bin/bash
echo "Welcome to Shell programming"
```

Output:

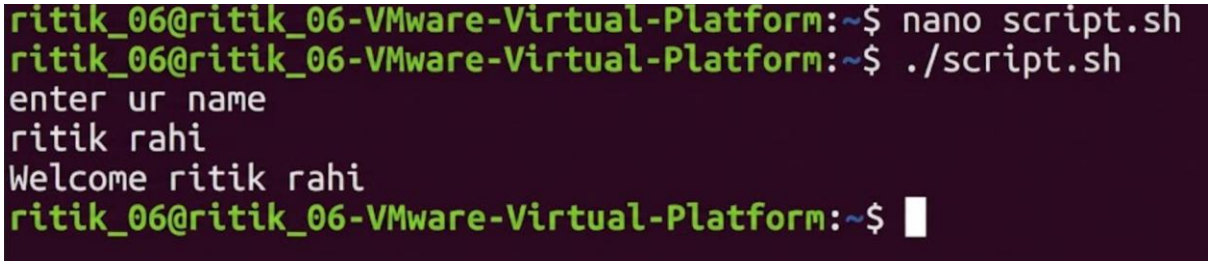


B) Input and Output using read

Program:

```
#!/bin/bash
echo "enter your name"
read name
echo "Welcome $name"
```

Output:

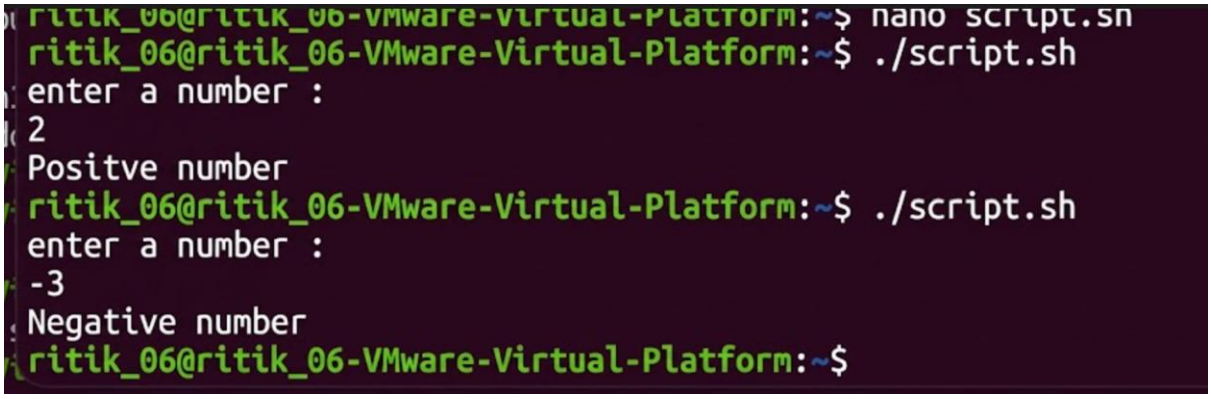


C) Check Positive or Negative using if-else

Program:

```
#!/bin/bash
echo "enter a number :"
read num
if [ $num -gt 0 ]
then
    echo "Positve number"
else
    echo "Negative number"
fi
```

Output:

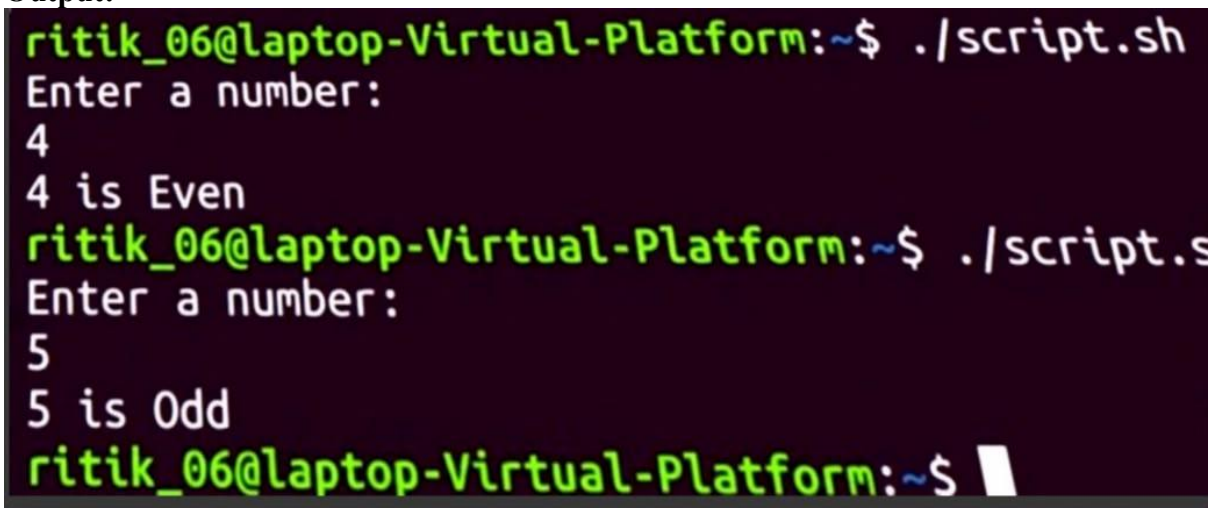


D) Check Even or Odd using if-else

Program:

```
#!/bin/bash
echo "Enter a number:"
read n
if [ $((n % 2)) -eq 0 ]
then
    echo "$n is Even"
else
    echo "$n is Odd"
fi
```

Output:



E) Login Check using if-else

Program:

```
#!/bin/bash
echo "Enter username:"
read user
echo "Enter password:"
read pass
if [ "$user" = "admin" ] && [ "$pass" = "1234" ]
then
    echo "Login Sucessfull"
else
    echo "invalid username or password"
```

```
fi
```

Output:



F) Looping using for (sh)

Program:

```
#!/bin/sh
for i in 1 2 3 4 5
do
    echo $i
done
```

Output:



G) Looping using for – brace range (bash)

Program:

```
#!/bin/bash
for i in {1..5}
do
    echo $i
done
```

Output:

```
$ ./script.sh
1
2
3
4
5
```

H) Looping using for – C-style (bash)

Program:

```
#!/bin/bash
for ((i=1; i<=5; i++))
do
    echo $i
done
```

Output:

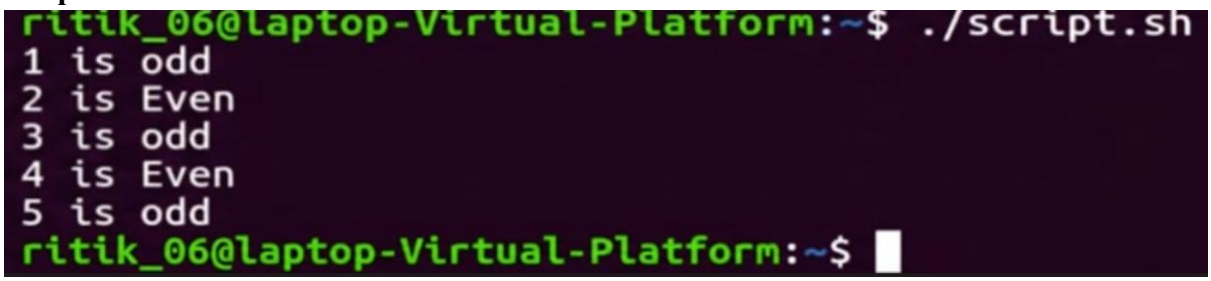
```
$ ./script.sh
1
2
3
4
5
```

I) Check Even or Odd using for

Program:

```
#!/bin/sh
for i in 1 2 3 4 5
do
    if [ $((i % 2)) -eq 0 ]
    then
        echo "$i is Even"
    else
        echo "$i is odd"
    fi
done
```

Output:



J) Print Multiplication Table

Program:

```
#!/bin/sh
echo "Enter a number:"
read n
for i in 1 2 3 4 5 6 7 8 9 10
do
    echo "$n x $i = $((n * i))"
done
```

Output:

```
ritik_06@laptop-Virtual-Platform:~$ ./script.sh
Enter a number:
4
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
4 x 10 = 40
ritik_06@laptop-Virtual-Platform:~$
```

K) Looping using while

Program:

```
#!/bin/bash
i=1
while [ $i -le 5 ]
do
    echo $i
    i=$((i+1))
done
```

Output:

```
ritik_06@laptop-Virtual-Platform:~$ nano script.sh
ritik_06@laptop-Virtual-Platform:~$ ./script.sh
1
2
3
4
5
ritik_06@laptop-Virtual-Platform:~$
```

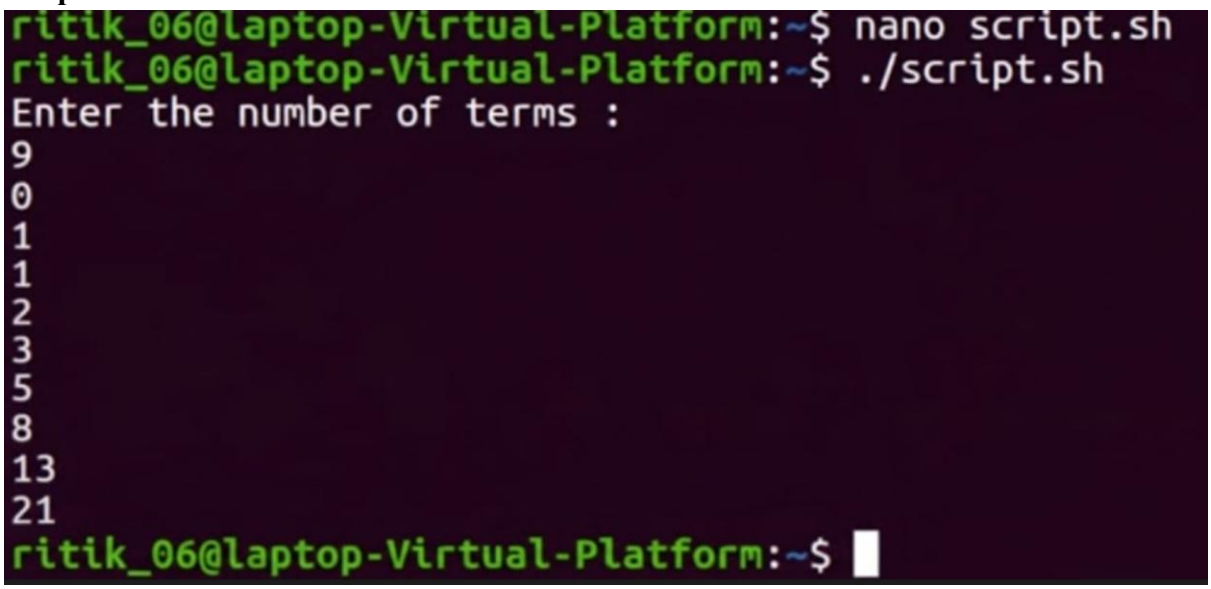
L) Fibonacci Series

Program:

```
#!/bin/sh
echo "Enter the number of terms :"
read n
a=0
b=1
i=1
while [ $i -le $n ]
do
    echo $a
    c=$((a + b))
    a=$b
    b=$c
```

```
    i=$((i + 1))
done
```

Output:



M) Sum of First N Numbers

Program:

```
#!/bin/sh
echo "Enter a number:"
read n
i=1
sum=0
while [ $i -le $n ]
do
    sum=$((sum + i))
    i=$((i + 1))
done
echo "Sum = $sum"
```

Output:



N) Factorial of a Number

Program:

```
#!/bin/sh
echo "Enter a number:"
read n
fact=1
```

```
i=1
while [ $i -le $n ]
do
    fact=$((fact * i))
    i=$((i + 1))
done
echo "Factorial=$fact"
```

Output:

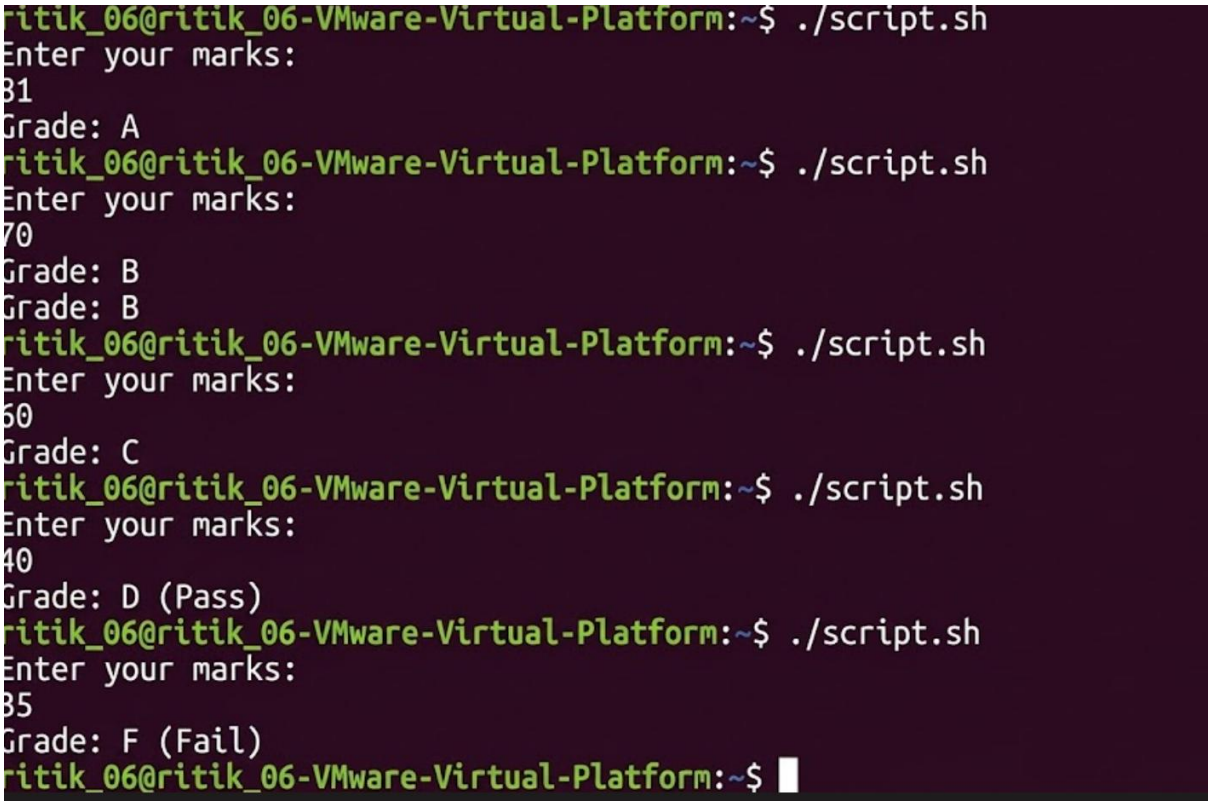


O) Multi-Level Branching using Nested if-else

Program:

```
#!/bin/bash
echo "Enter your marks:"
read marks
if [ $marks -ge 90 ]
then
    echo "Grade: A+"
elif [ $marks -ge 80 ]
then
    echo "Grade: A"
elif [ $marks -ge 70 ]
then
    echo "Grade: B"
elif [ $marks -ge 60 ]
then
    echo "Grade: C"
elif [ $marks -ge 40 ]
then
    echo "Grade: D (Pass)"
else
    echo "Grade: F (Fail)"
fi
```

Output:

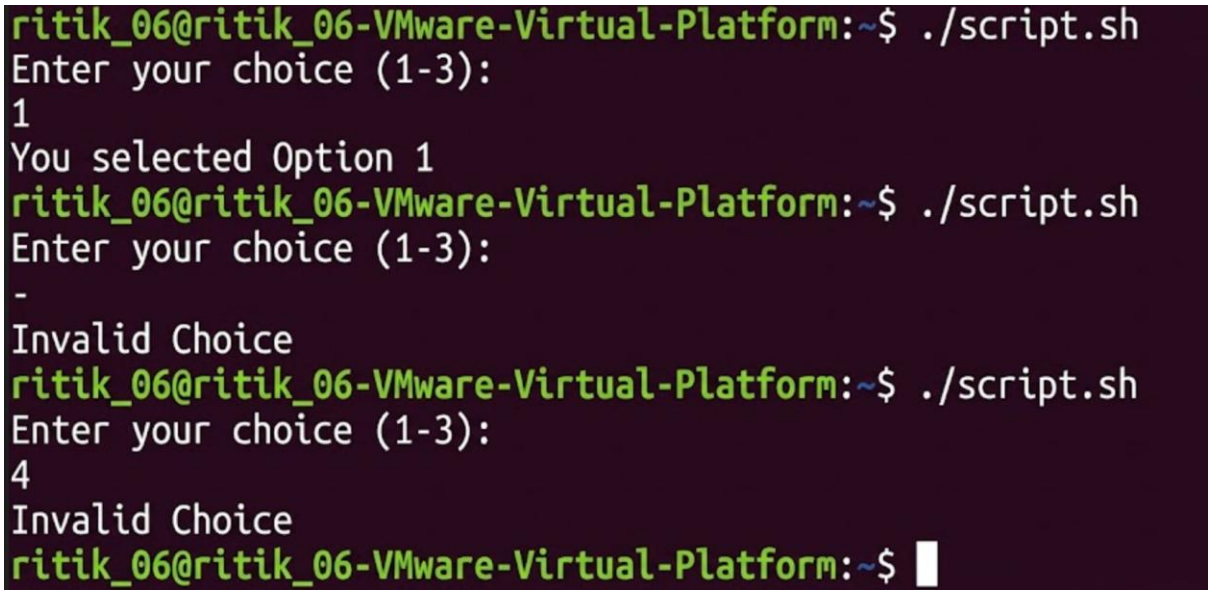


P) Multi-Level Branching using case

Program:

```
#!/bin/bash
echo "Enter your choice (1-3):"
read choice
case $choice in
    1) echo "You selected Option 1";;
    2) echo "You selected Option 2";;
    3) echo "You selected Option 3";;
    *) echo "Invalid Choice";;
esac
```

Output:



Result: Shell scripts using input/output statements, decision-making, looping and multi-level branching were written and executed successfully in Linux.

Practical No. 3: Process Creation using fork() – Parent, Child, Orphan and Zombie Processes

✓ **Theory/Description:**

A **process** is a program in execution. Each process has a unique **Process ID (PID)** and also stores the ID of the process that created it, the **Parent Process ID (PPID)**. In Linux, a new process is created with the `fork()` system call, which makes an almost exact copy of the calling process. After fork(), both processes continue from the next statement, and the return value tells them apart: a negative value means creation failed, **0** is returned in the child, and the **child's PID** is returned in the parent. `getpid()` and `getppid()` give the PID and PPID of the current process.

If the parent finishes before the child, the child becomes an **orphan process** and is adopted by the init/systemd process (PID 1), so its PPID changes. If the child finishes but the parent has not yet collected its exit status using `wait()`, the child remains in the process table as a **zombie process**, shown with state **Z** (`<defunct>`) in `ps`.

Commands Used

Sr. No.	Command	Purpose	Example
1	nano	Create/edit a C program	nano fork.c
2	gcc	Compile the C program	gcc fork.c -o fork
3	./program	Execute the compiled program	./fork
4	ls	Verify the executable file	ls
5	ps	Display process status	ps -el

Procedure

1. Open Terminal (Ctrl + Alt + T).
2. Update package list: `sudo apt update`
3. Install GCC: `sudo apt install build-essential`
4. Verify GCC: `gcc --version`
5. Create the program: `nano fork.c`
6. Write the program, save (Ctrl + O, Enter) and exit (Ctrl + X).
7. Compile: `gcc fork.c -o fork`
8. Run: `./fork`

A) Parent and Child Process

Aim: To study process creation in Linux using the fork() system call and observe the Parent and Child processes.

Program:

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;
    pid = fork();

    if (pid < 0)
```

```
    {
        printf("Process creation failed.\n");
    }
    else if (pid == 0)
    {
        printf("\n----- Child Process -----\n");
        printf("Child Process ID (PID): %d\n", getpid());
        printf("Parent Process ID (PPID): %d\n", getppid());
        printf("Child process is executing...\n");
        printf("Child process completed.\n");
    }
    else
    {
        printf("\n----- Parent Process -----\n");
        printf("Parent Process ID (PID): %d\n", getpid());
        printf("Child Process ID: %d\n", pid);
        printf("Parent process is executing...\n");
        printf("Parent process completed.\n");
    }
    return 0;
}
```

Output:

```
ritik_06@ritik_06-VMware-Virtual-Platform:~$ ./fork


----- Child Process -----
Child Process ID (PID): 5814
Parent Process ID (PPID): 5813
Child process is execu0ng...
Child process completed.
----- Parent Process -----
Parent Process ID (PID): 5813
Child Process ID: 5814
Parent process is execu0ng...
```

B) Orphan Process

Aim: To create and observe an Orphan Process using the fork() system call.

Program:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    pid_t pid = fork();

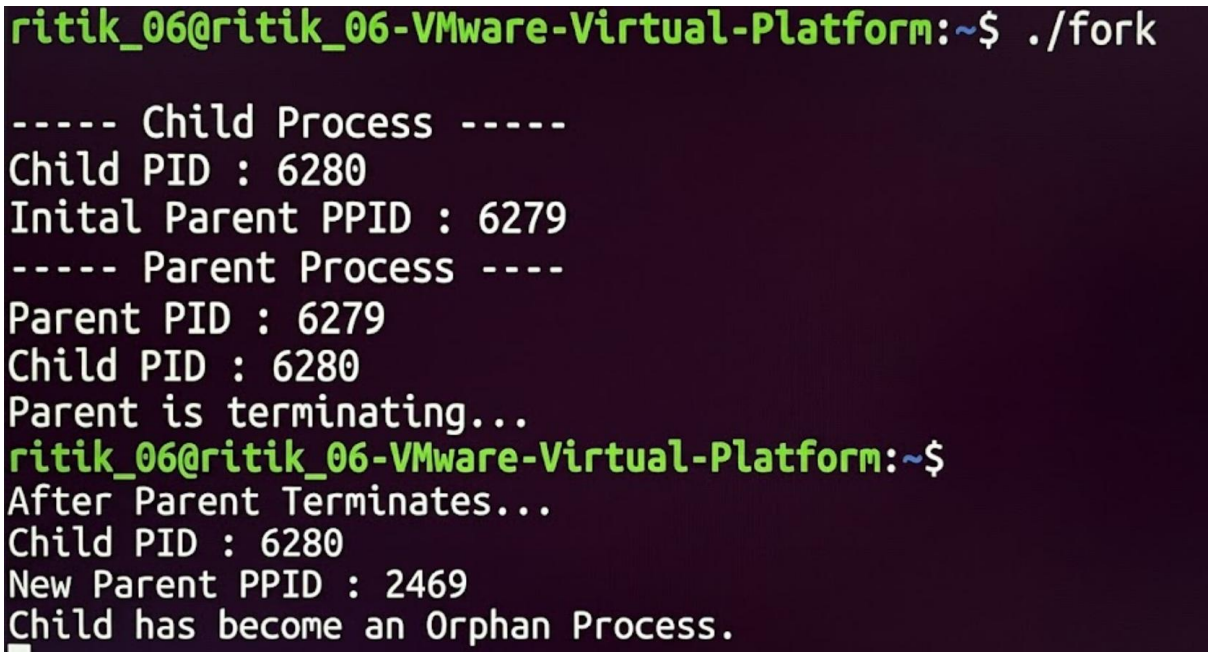
    if (pid < 0)
    {
        printf("Fork failed.\n");
```

```
    }
    else if (pid == 0)
    {
        printf("\n----- Child Process -----\n");
        printf("Child PID          : %d\n", getpid());
        printf("Initial Parent PPID : %d\n", getppid());

        sleep(5); // Wait for parent to terminate

        printf("\nAfter Parent Terminates...\n");
        printf("Child PID       : %d\n", getpid());
        printf("New Parent PPID : %d\n", getppid());
        printf("Child has become an Orphan Process.\n");
    }
    else
    {
        printf("\n----- Parent Process -----\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);
        printf("Parent is terminating...\n");
        exit(0);
    }
    return 0;
}
```

Output:



C) Zombie Process

Aim: To create and observe a Zombie Process using the fork() system call.

1. Create the file: `nano zombie.c` and write the program.
2. Compile: `gcc zombie.c -o zombie`
3. Run: `./zombie`
4. Open another terminal and check the zombie process: `ps -el | grep Z`

Program:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    pid_t pid;
    pid = fork();

    if (pid < 0)
    {
        printf("Process creation failed.\n");
    }
    else if (pid == 0)
    {
        printf("\n----- Child Process -----\n");
        printf("Child Process ID (PID): %d\n", getpid());
        printf("Child process is terminating...\n");
        exit(0);
    }
    else
    {
        printf("\n----- Parent Process -----\n");
        printf("Parent Process ID (PID): %d\n", getpid());
        printf("Child Process ID: %d\n", pid);
        printf("Parent process is sleeping...\n");
        sleep(15);
        printf("Parent process completed.\n");
    }
    return 0;
}
```

Output:

```
ritik_06@ritik_06-VMware-Virtual-Platform:~$ nano zombie.c
ritik_06@ritik_06-VMware-Virtual-Platform:~$ gcc zombie.c -o zombie
ritik_06@ritik_06-VMware-Virtual-Platform:~$ ./zombie

----- Parent Process -----
Parent Process ID (PID): 6558
Child Process ID (PID): 6559
Parent process is sleeping...

----- Child Process -----
Child Process ID (PID): 6559
Child process is terminating...
Parent process completed.
ritik_06@ritik_06-VMware-Virtual-Platform:~$ ps -el | grep Z
F S   UID          PID    PPID  C PRI   NI ADDR SZ WCHAN    TTY
ritik_06@ritik_06-VMware-Virtual-Platform:~$
```

Result: Process creation using fork() was studied, and parent, child, orphan and zombie processes were created and observed successfully.

Practical No. 4: CPU Scheduling Algorithms – FCFS, SJF, Priority and Round Robin

✓ **Theory/Description:**

When several processes are ready to run, the **CPU scheduler** decides which one gets the CPU next. In **non-preemptive** scheduling a process keeps the CPU until it finishes, whereas in **preemptive** scheduling the CPU can be taken away from it. The algorithms are compared using **Turnaround Time (TAT = CT − AT)** and **Waiting Time (WT = TAT − BT)**, where AT is arrival time, BT is burst time and CT is completion time. A **Gantt chart** shows which process runs in each time interval.

- **FCFS** runs processes in the order they arrive. It is simple but short processes may wait behind a long one (convoy effect).
- **SJF** runs the process with the shortest burst time first and gives the minimum average waiting time, but long processes may starve.
- **Priority** runs the process with the highest priority first (a smaller number means higher priority here). Starvation is avoided using aging.
- **Round Robin** gives each process a fixed time quantum in turn and is preemptive and fair, which suits time-sharing systems.

A) FCFS Scheduling

Aim: Write a program to implement FCFS CPU Scheduling and find the Average Waiting Time and Average Turnaround Time along with the Gantt Chart.

Code:

```
process = ["P1", "P2", "P3", "P4"]
at = [0, 2, 4, 6]   # Arrival Time
bt = [1, 3, 8, 6]   # Burst Time
n = len(process)

wt = [0] * n
tat = [0] * n
ct = [0] * n

# First process
ct[0] = at[0] + bt[0]
tat[0] = ct[0] - at[0]
wt[0] = tat[0] - bt[0]

# Remaining processes
for i in range(1, n):
    if ct[i-1] < at[i]:
        ct[i] = at[i] + bt[i]   # CPU Idle
    else:
        ct[i] = ct[i-1] + bt[i]
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]

# Display Table
print("Process\tAT\tBT\tCT\tTAT\tWT")
for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t", ct[i], "\t", tat[i], "\t", wt[i])

print("\nAverage Waiting Time =", sum(wt) / n)
print("Average Turnaround Time =", sum(tat) / n)
```

```
# Gantt Chart
print("\nGantt Chart:")
time = 0
print(time, end="")
for i in range(n):
    if time < at[i]:
        print(f" --Idle-- {at[i]}", end="")
        time = at[i]
    print(f" --{process[i]}-- {ct[i]}", end="")
    time = ct[i]
print()
```

Output:

```
>>>
    ===== RESTART: C:/Users/tanis/Documents/ritik_06/CollegeDOC/osv/prac4 sjf.py ====
Process AT      BT      CT      TAT     WT
P1      0       2       2       2       0
P2      2       4       7       5       1
P3      2       1       3       1       0
P4      5       2       9       4       2

Average Waiting Time = 0.75
Average Turnaround Time = 3.0

Gantt Chart:
0 --P1-- 2 --P3-- 3 --P2-- 7 --P4-- 9
```

B) SJF Scheduling

Aim: Write a program to implement SJF CPU Scheduling and find the Average Waiting Time and Average Turnaround Time along with the Gantt Chart.

Code:

```
process = ["P1", "P2", "P3", "P4"]
at = [0, 2, 2, 5]   # Arrival Time
bt = [2, 4, 1, 2]   # Burst Time
n = len(process)

wt = [0] * n
tat = [0] * n
ct = [0] * n
completed = [False] * n
time = 0
completed_process = 0
gantt = []

while completed_process < n:
    shortest = -1
    min_bt = 999
    for i in range(n):
        if at[i] <= time and completed[i] == False:
            if bt[i] < min_bt:
                min_bt = bt[i]
                shortest = i
    if shortest == -1:
        gantt.append(("Idle", time))
        time += 1
        continue
    time = time + bt[shortest]
    ct[shortest] = time
    tat[shortest] = ct[shortest] - at[shortest]
    wt[shortest] = tat[shortest] - bt[shortest]
```

```
    completed[shortest] = True
    completed_process += 1
    gantt.append((process[shortest], time))

print("Process\tAT\tBT\tCT\tTAT\tWT")
for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i],
          "\t", ct[i], "\t", tat[i], "\t", wt[i])

print("\nAverage Waiting Time =", sum(wt) / n)
print("Average Turnaround Time =", sum(tat) / n)

print("\nGantt Chart:")
start = 0
print(start, end="")
for p, end_time in gantt:
    print(f" --{p}-- {end_time}", end="")
print()
```

Output:

```
>>>
    ===== RESTART: C:/Users/tanis/Documents/ritik_06/CollegeDOC/osv/prac4 sjf.py ====
Process AT    BT    CT    TAT    WT
P1      0     1     1     1      0
P2      2     3     5     3      0
P3      4     8     13    9      1

Average Waiting Time = 2.0
Average Turnaround Time = 6.5

Gantt Chart:
0 --P1-- 1 --Idle-- 2 --P2-- 5 --P3-- 13 --P4-- 19
```

C) Priority Scheduling

Aim: Write a program to implement Priority CPU Scheduling and find the Average Waiting Time and Average Turnaround Time along with the Gantt Chart.

Code:

```
process = ["P1", "P2", "P3", "P4"]
at = [0, 2, 3, 5]
bt = [7, 4, 1, 2]
priority = [2, 1, 4, 2]
n = len(process)
```

```
wt = [0] * n
tat = [0] * n
ct = [0] * n
completed = [False] * n
time = 0
count = 0
gantt = []

while count < n:
    highest = -1
    min_priority = 999
    for i in range(n):
        if at[i] <= time and completed[i] == False:
            if priority[i] < min_priority:
                min_priority = priority[i]
                highest = i
    if highest == -1:
        gantt.append(("Idle", time))
        time += 1
        continue
    time += bt[highest]
    ct[highest] = time
    tat[highest] = ct[highest] - at[highest]
    wt[highest] = tat[highest] - bt[highest]
    completed[highest] = True
    count += 1
    gantt.append((process[highest], time))

print("Process\tAT\tBT\tPriority\tCT\tTAT\tWT")
for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t",
          priority[i], "\t\t", ct[i], "\t", tat[i], "\t", wt[i])

print("\nAverage Waiting Time =", sum(wt) / n)
print("Average Turnaround Time =", sum(tat) / n)

print("\nGantt Chart:")
time = 0
print(time, end="")
for p, end_time in gantt:
    print(f" --{p}-- {end_time}", end="")
print()
```

Output:

```
==== RESTART: C:/Users/tanis/Documents/ritik_06/CollegDOC/osv/prac  4 cpus
y =======
Process AT      BT      Priority        CT      TAT     WT
P1      0       7       2               7       7       0
P2      2       4       1               11      9       5
P3      3       1       4               14      11      10
P4      5       2       2               13      8       6

Average Waiting Time = 5.25
Average Turnaround Time = 8.75

Gantt Chart:
0 --P1-- 7 --P2-- 11 --P4-- 13 --P3-- 14
```

D) Round Robin Scheduling

Aim: Write a program to implement Round robin CPU Scheduling and find the Average Waiting Time and Average Turnaround Time along with the Gantt Chart.

Code:

```
process = ["P1", "P2", "P3", "P4"]
at = [0, 1, 3, 5]
bt = [5, 4, 2, 6]
tq = 2
n = len(process)

rt = bt[:]
ct = [0] * n
tat = [0] * n
wt = [0] * n
queue = []
visited = [False] * n
time = 0
completed = 0
gantt = []

queue.append(0)
visited[0] = True

while completed < n:
    if len(queue) == 0:
        gantt.append(("Idle", time))
        time += 1
        for i in range(n):
            if at[i] <= time and not visited[i]:
                queue.append(i)
                visited[i] = True
        continue

    i = queue.pop(0)
    if rt[i] > tq:
        time += tq
        rt[i] -= tq
    else:
        time += rt[i]
        rt[i] = 0
        ct[i] = time
        completed += 1
    gantt.append((process[i], time))

    for j in range(n):
        if at[j] <= time and not visited[j]:
            queue.append(j)
            visited[j] = True
    if rt[i] > 0:
        queue.append(i)

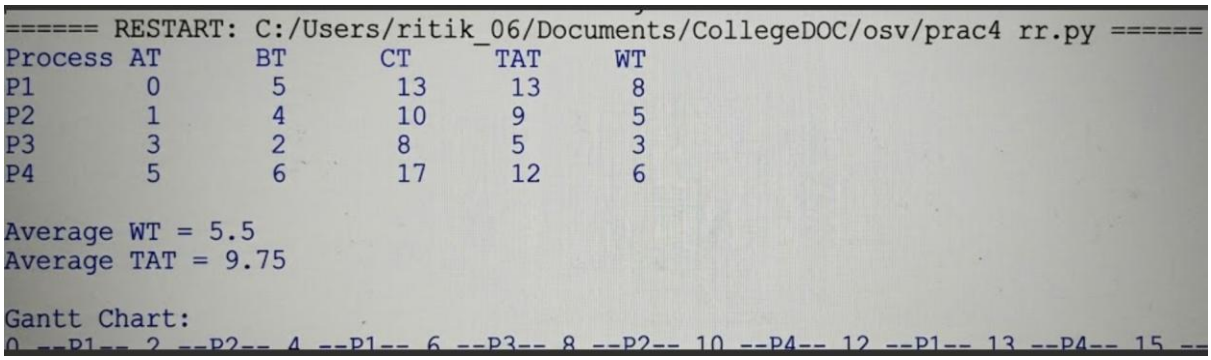
for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]
```

```
print("Process\tAT\tBT\tCT\tTAT\tWT")
for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t",
          ct[i], "\t", tat[i], "\t", wt[i])

print("\nAverage WT =", sum(wt) / n)
print("Average TAT =", sum(tat) / n)

print("\nGantt Chart:")
print("0", end="")
for p, t in gantt:
    print(f" --{p}-- {t}", end="")
print()
```

Output:



Result: FCFS, SJF, Priority and Round Robin CPU scheduling algorithms were implemented, and the average waiting time, average turnaround time and Gantt chart were obtained for each.

Practical No. 5: Banker's Algorithm for Deadlock Avoidance

Aim: Simulation of Banker's algorithm to check whether the given system is in a safe state or not. Also check whether an additional resource request can be granted immediately.

- ✓ **Theory/Description:**

A **deadlock** occurs when a set of processes wait forever, each holding a resource needed by another. It can happen only when mutual exclusion, hold and wait, no preemption and circular wait all hold together. The **Banker's algorithm** avoids deadlock by granting a request only if the system remains in a **safe state**, that is, if there is at least one order (safe sequence) in which every process can obtain its remaining need and finish.

It uses the **Available**, **Max** and **Allocation** matrices and calculates **Need = Max − Allocation**. The safety algorithm starts with Work = Available, repeatedly picks an unfinished process whose Need ≤ Work, adds its Allocation back to Work, and marks it finished. If all processes finish, the system is safe. A new request is granted only if Request ≤ Need and Request ≤ Available.

Code:

```
allocation = [
    [0, 1, 0],
    [2, 0, 0],
    [3, 0, 2],
    [2, 1, 1],
    [0, 0, 2]
]

maximum = [
    [7, 5, 3],
    [3, 2, 2],
    [9, 0, 2],
    [2, 2, 2],
    [4, 3, 3]
]

available = [3, 3, 2]

n = 5
m = 3

# Calculate Need Matrix
need = []
for i in range(n):
    row = []
    for j in range(m):
        row.append(maximum[i][j] - allocation[i][j])
    need.append(row)

print("Need Matrix:")
for i in range(n):
    print("P", i, ":", need[i])

# Safety Algorithm
work = available[:]
finish = [False] * n
safe = []

while len(safe) < n:
    found = False
```

```
    for i in range(n):
        if not finish[i]:
            possible = True
            for j in range(m):
                if need[i][j] > work[j]:
                    possible = False
                    break
            if possible:
                for j in range(m):
                    work[j] += allocation[i][j]
                finish[i] = True
                safe.append(i)
                found = True
    if not found:
        break

if len(safe) == n:
    print("\nSystem is in Safe State")
    print("Safe Sequence:", end=" ")
    for p in safe:
        print("P" + str(p), end=" ")
else:
    print("\nSystem is NOT in Safe State")

# ---------------- Additional Resource Request ----------------
print("\n\nAdditional Resource Request")
process = int(input("Enter Process Number: "))
request = list(map(int, input("Enter Request (3 values): ").split()))

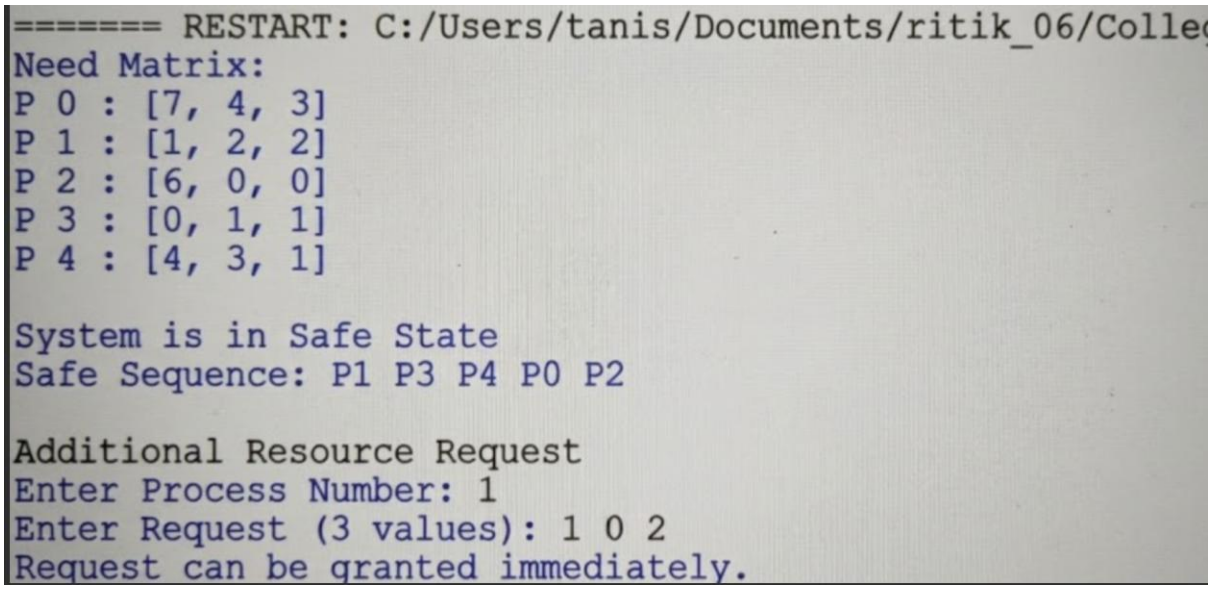
grant = True

# Check Request <= Need
for i in range(m):
    if request[i] > need[process][i]:
        print("Request exceeds Need.")
        grant = False
        break

# Check Request <= Available
if grant:
    for i in range(m):
        if request[i] > available[i]:
            print("Resources are not available.")
            grant = False
            break

if grant:
    print("Request can be granted immediately.")
else:
    print("Request cannot be granted.")
```

Output:



Result: Banker's algorithm was simulated successfully. The system is in a safe state with safe sequence P1, P3, P4, P0, P2, and the request (1, 0, 2) by P1 can be granted immediately.

Practical No. 6: Parallel Thread Management using POSIX Threads (pthread)

Aim: Parallel Thread Management using the POSIX Thread (pthread) Library (Data Parallelism).

- ✓ **Theory/Description:**

A **thread** is a lightweight unit of execution within a process. Threads of the same process share code, global data and open files, but each has its own stack and registers, which makes them faster to create and switch than processes. In **data parallelism**, the same task is performed on different parts of the data at the same time; here, one thread searches the first half of the array and another searches the second half.

POSIX threads (pthreads) is the standard thread library on Linux, used through `<pthread.h>` and compiled with the `-pthread` flag. `pthread_create()` starts a new thread, `pthread_join()` makes the main program wait for a thread to finish, and `pthread_exit()` ends a thread. The order of thread output may vary between runs because it depends on the scheduler.

Commands Used

Sr. No.	Command	Purpose	Example
1	nano	Create/edit a C program	nano search.c
2	gcc	Compile the C program with pthread library	gcc search.c -o search -pthread
3	./program	Execute the compiled program	./search
4	ls	Verify the executable file	ls

Procedure

1. Open the Terminal (Ctrl + Alt + T).
2. Create a C program: `nano search.c`
3. Type the program and save the file.
4. Compile the program: `gcc search.c -o search -pthread`
5. Execute the program: `./search`

Program:

```
#include <stdio.h>
#include <pthread.h>

int arr[] = {5, 10, 15, 20, 25, 30, 35, 40, 45, 50};
int key = 25;

void *searchFirstHalf(void *arg)
{
    for (int i = 0; i < 5; i++)
    {
        if (arr[i] == key)
        {
            printf("Thread 1: Element %d found at index %d\n", key, i);
            pthread_exit(NULL);
        }
    }
    printf("Thread 1: Element not found.\n");
    pthread_exit(NULL);
```

```
}

void *searchSecondHalf(void *arg)
{
    for (int i = 5; i < 10; i++)
    {
        if (arr[i] == key)
        {
            printf("Thread 2: Element %d found at index %d\n", key, i);
            pthread_exit(NULL);
        }
    }
    printf("Thread 2: Element not found.\n");
    pthread_exit(NULL);
}

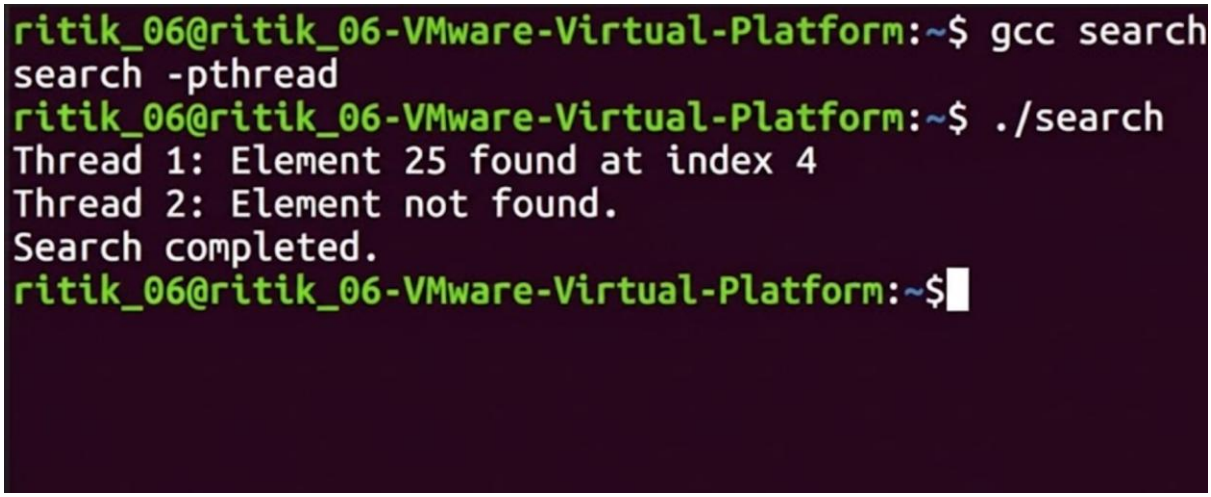
int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, searchFirstHalf, NULL);
    pthread_create(&t2, NULL, searchSecondHalf, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Search completed.\n");
    return 0;
}
```

Output:



Result: Parallel searching of an array was implemented using two POSIX threads, and the element 25 was found at index 4 by thread 1.

Practical No. 7: Dynamic Memory Allocation – First Fit, Best Fit and Worst Fit

✓ **Theory/Description:**

In **dynamic (variable) partitioning**, memory is given to a process according to its size, and the free areas of memory are called holes. When a process requests memory, the operating system chooses a hole using one of three strategies. **First Fit** uses the first hole that is large enough and is the fastest. **Best Fit** uses the smallest hole that is large enough, leaving the least space unused. **Worst Fit** uses the largest hole so that the leftover space is still usable. Repeated allocation leaves small scattered holes, which causes **external fragmentation**; space left unused inside an allocated block is **internal fragmentation**.

Memory blocks used: 20, 100, 40, 200, 10. Process sizes: 90, 50, 30, 40.

A) First Fit

Aim: Demonstrate the First-fit dynamic memory allocation algorithm.

Code:

```
blocks = [20, 100, 40, 200, 10]
processes = [90, 50, 30, 40]

allocation = [-1] * len(processes)

for i in range(len(processes)):
    for j in range(len(blocks)):
        if blocks[j] >= processes[i]:
            allocation[i] = j
            blocks[j] -= processes[i]
            break

print("Process\tSize\tBlock")
for i in range(len(processes)):
    if allocation[i] != -1:
        print(f"P{i+1}\t{processes[i]}\tBlock {allocation[i]+1}")
    else:
        print(f"P{i+1}\t{processes[i]}\tNot Allocated")
```

Output:

```
>
== RESTART: C:/Users/tanis/Documents/ritik_06/CollegeDOC/osv/prac 7 firstfit.py
Process Size     Block
P1      90       Block 2
P2      50       Block 4
P3      30       Block 3
P4      40       Block 4
```

B) Best Fit

Aim: Demonstrate the Best-fit dynamic memory allocation algorithm.

Code:

```
blocks = [20, 100, 40, 200, 10]
processes = [90, 50, 30, 40]

allocation = [-1] * len(processes)

for i in range(len(processes)):
    best = -1
    for j in range(len(blocks)):
        if blocks[j] >= processes[i]:
            if best == -1 or blocks[j] < blocks[best]:
                best = j
    if best != -1:
        allocation[i] = best
        blocks[best] -= processes[i]

print("Process\tSize\tBlock")
for i in range(len(processes)):
    if allocation[i] != -1:
        print(f"P{i+1}\t{processes[i]}\tBlock {allocation[i]+1}")
    else:
        print(f"P{i+1}\t{processes[i]}\tNot Allocated")
```

Output:

```
File Edit Shell Debug Options Window Help
>>>
=== RESTART: C:/Users/ritik_06/Documents/CollegeDOC/osv/prac4 sjf.py =
Process Size    Block
P1          90  Block 2
P2          50  Block 4
P3          30  Block 3
P4          40  Block 4
```

C) Worst Fit

Aim: Demonstrate the Worst-fit dynamic memory allocation algorithm.

Code:

```
blocks = [20, 100, 40, 200, 10]
processes = [90, 50, 30, 40]

allocation = [-1] * len(processes)

for i in range(len(processes)):
    worst = -1
    for j in range(len(blocks)):
        if blocks[j] >= processes[i]:
            if worst == -1 or blocks[j] > blocks[worst]:
                worst = j
    if worst != -1:
        allocation[i] = worst
        blocks[worst] -= processes[i]

print("Process\tSize\tBlock")
for i in range(len(processes)):
    if allocation[i] != -1:
        print(f"P{i+1}\t{processes[i]}\tBlock {allocation[i]+1}")
    else:
        print(f"P{i+1}\t{processes[i]}\tNot Allocated")
```

Output:

```
File  Edit  Shell  Debug  Options  Window  Help
== RESTART: C:/Users/tanis/Documents/ritik_06/CollegeDOC/osv/prac4 sjf.py ==
Process    Size    Block
P1         90      Block 4
P2         50      Block 4
P3         30      Block 2
P4         40      Block 2
```

Result: First Fit, Best Fit and Worst Fit memory allocation algorithms were implemented and the block allocated to each process was displayed.

Practical No. 8: Page Replacement Algorithms – FIFO, LRU and Optimal

✓ **Theory/Description:**

In **paging**, a process is divided into fixed-size pages that are loaded into memory frames only when needed. When a referenced page is not in memory a **page fault** occurs, and if all frames are full a **page replacement algorithm** decides which page to remove. **FIFO** removes the oldest page in memory and can suffer from Belady's anomaly, where more frames give more faults. **LRU** removes the page that has not been used for the longest time. **Optimal** removes the page that will not be used for the longest time in the future; it gives the fewest page faults but needs future knowledge, so it is used as a benchmark.

Reference string used: 1, 2, 3, 1, 4, 5, 2, 1, 2, 3, 4, 5 with 3 frames.

A) FIFO Page Replacement

Aim: Demonstrate the FIFO Page Replacement Algorithm.

Code:

```
pages = [1, 2, 3, 1, 4, 5, 2, 1, 2, 3, 4, 5]
frames = 3
memory = []
page_faults = 0

for page in pages:
    if page not in memory:
        page_faults += 1
        if len(memory) < frames:
            memory.append(page)
        else:
            memory.pop(0)
            memory.append(page)
    print(page, "->", memory)

print("\nTotal Page Faults:", page_faults)
```

Output:

```
===== RESTART: C:/Users/tanis/Documents/ritik_06/CollegeDOC/osv/prac 8 lru.py ==
1 -> [1]
2 -> [1, 2]
3 -> [1, 2, 3]
1 -> [1, 2, 3]
4 -> [2, 3, 4]
5 -> [3, 4, 5]
2 -> [4, 5, 2]
1 -> [5, 2, 1]
2 -> [5, 2, 1]
3 -> [2, 1, 3]
4 -> [1, 3, 4]
5 -> [3, 4, 5]

Total Page Faults: 10
```

B) LRU Page Replacement

Aim: Demonstrate the LRU Page Replacement Algorithm.

Code:

```
pages = [1, 2, 3, 1, 4, 5, 2, 1, 2, 3, 4, 5]
frames = 3
memory = []
page_faults = 0

for page in pages:
    if page not in memory:
        page_faults += 1
        if len(memory) < frames:
            memory.append(page)
        else:
            memory.pop(0)
            memory.append(page)
    else:
        memory.remove(page)
        memory.append(page)
    print(page, "->", memory)

print("\nTotal Page Faults:", page_faults)
```

Output:

```
===== RESTART: C:/Users/tanis/Documents/ritik_06/CollegeDOC/osv/prac8 sjf.py =
1 -> [1]
2 -> [1, 2]
3 -> [1, 2, 3]
1 -> [1, 2, 3]
4 -> [2, 3, 4]
5 -> [3, 4, 5]
2 -> [4, 5, 2]
1 -> [5, 2, 1]
2 -> [5, 2, 1]
3 -> [2, 1, 3]
4 -> [1, 3, 4]
5 -> [3, 4, 5]

Total Page Faults: 10
```

C) Optimal Page Replacement

Aim: Demonstrate the Optimal Page Replacement Algorithm.

Code:

```
pages = [1, 2, 3, 1, 4, 5, 2, 1, 2, 3, 4, 5]
frames = 3
memory = []
page_faults = 0

for i in range(len(pages)):
    page = pages[i]
    if page not in memory:
        page_faults += 1
        if len(memory) < frames:
            memory.append(page)
        else:
            future = pages[i + 1:]
            index = -1
            farthest = -1
            for j in range(len(memory)):
                if memory[j] not in future:
                    index = j
                    break
                position = future.index(memory[j])
                if position > farthest:
                    farthest = position
                    index = j
            memory[index] = page
    print(page, "->", memory)

print("\nTotal Page Faults:", page_faults)
```

Output:

```
$ python3 optimal.py
1 -> [1]
2 -> [1, 2]
3 -> [1, 2, 3]
1 -> [1, 2, 3]
4 -> [1, 2, 4]
5 -> [1, 2, 5]
2 -> [1, 2, 5]
1 -> [1, 2, 5]
2 -> [1, 2, 5]
3 -> [3, 2, 5]
4 -> [4, 2, 5]
5 -> [4, 2, 5]

Total Page Faults: 7
```

Result: FIFO, LRU and Optimal page replacement algorithms were implemented, giving 10, 10 and 7 page faults respectively for the given reference string.

Practical No. 9: Type-I and Type-II Hypervisors and VM Setup using a Type-II Hypervisor

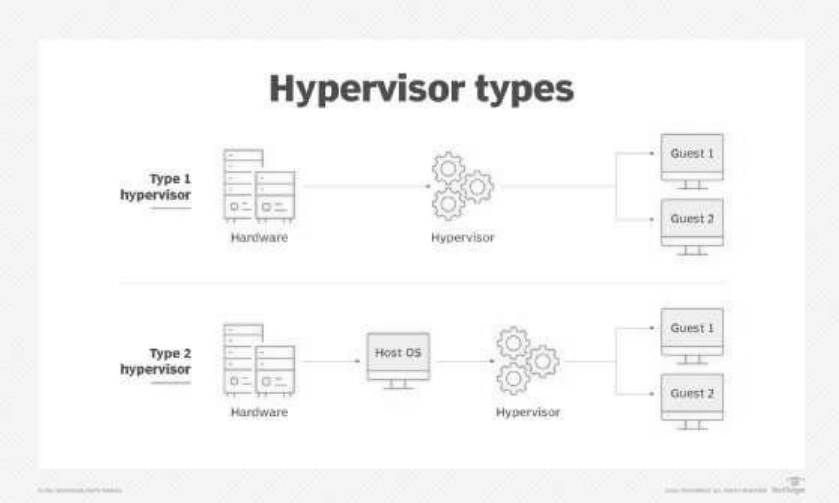
Aim: To study Type-I and Type-II hypervisors and set up a virtual machine using a Type-II hypervisor.

- ✓ **Theory/Description:**

A **hypervisor**, also called a Virtual Machine Monitor, is software that creates and manages virtual machines (VMs) by sharing the physical CPU, memory, storage and network among them. There are two main types:

1. Type-I Hypervisor (Bare-Metal) runs directly on the physical hardware and does not require a host operating system. It gives better performance and is commonly used in servers and data centers. Examples: VMware ESXi, Microsoft Hyper-V, Xen.

2. Type-II Hypervisor (Hosted) runs on top of an existing operating system, which manages the hardware, while the hypervisor creates and manages virtual machines. It is mainly used on desktops for learning and testing. Examples: VMware Workstation, Oracle VirtualBox.



Virtualization support (Intel VT-x / AMD-V) should be enabled in the BIOS of the host machine.

Steps

Step 1: Open VMware Workstation

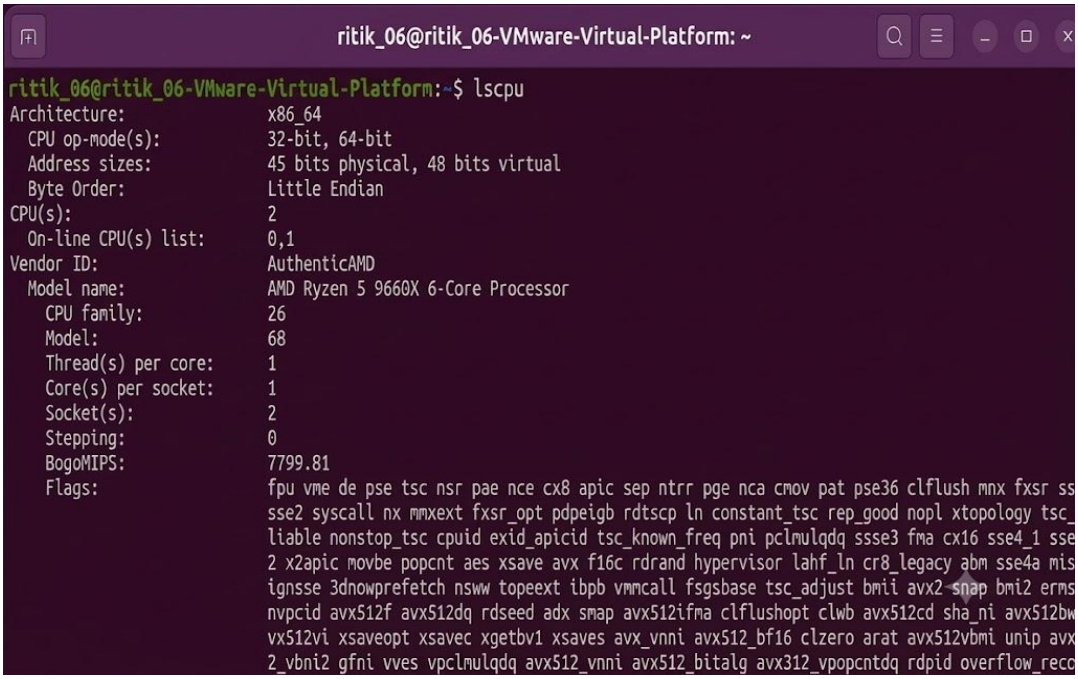
1. Open VMware Workstation 17 Player.
2. Click Create a New Virtual Machine.

Step 2: Select Ubuntu ISO

1. Select Installer disc image file (ISO).
2. Browse and select the Ubuntu ISO file.
3. Click Next.

Step 3: Configure the Virtual Machine

1. Enter the VM name and location.
2. Set the disk size (e.g. 25 GB).
3. Use Customize Hardware to set Memory (e.g. 4 GB), Processors (e.g. 2) and Network (NAT).



Step 4: Create the Virtual Machine

1. Review the configuration.
2. Click Finish.
3. The Ubuntu virtual machine will be created.

Step 5: Start the Virtual Machine

1. Select the Ubuntu VM.
2. Click Play Virtual Machine.
3. Ubuntu will start from the ISO file.

Step 6: Install Ubuntu

1. Select Install Ubuntu.
2. Follow the installation instructions.
3. Create a username and password.
4. Complete the installation.
5. Restart the virtual machine when prompted.

Commands for Verification

Open the Ubuntu Terminal and execute:

1. `lscpu` – displays the CPU information available to the virtual machine.
2. `free -h` – displays the total, used and available RAM.
3. `uname -a` – displays information about the Linux kernel and system.





Result: The Type-I and Type-II hypervisors were studied, and a virtual machine was successfully created and executed using the Type-II hypervisor VMware Workstation.

Practical No. 10: Operating System Virtualization – Ubuntu VM using VMware Workstation

Aim: To implement operating system virtualization by creating and running an Ubuntu virtual machine using VMware Workstation.

✓ **Theory/Description:**

Virtualization is a technique that allows multiple virtual computers or operating systems to run on a single physical computer. A **Virtual Machine (VM)** behaves like an independent computer, with its own virtual CPU, RAM, storage and network resources. The physical machine is called the host and the operating system inside the VM is called the guest. Virtualization reduces hardware cost, keeps each VM isolated, and allows snapshots, so different operating systems can be tested safely. In this practical, VMware Workstation is used to create an Ubuntu virtual machine on a Windows host system.

Steps

Step 1: Open VMware Workstation

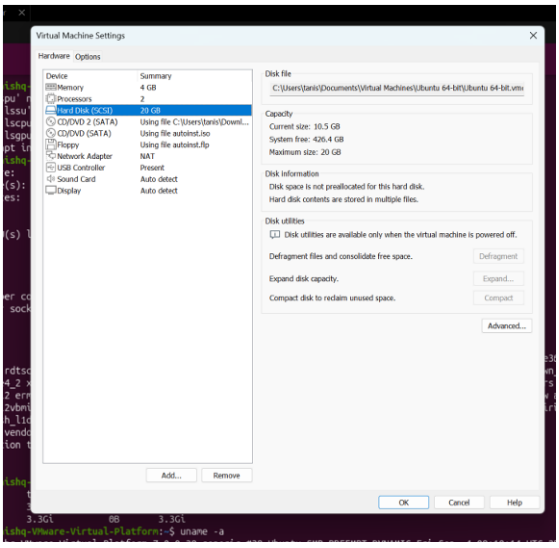
1. Open VMware Workstation 17 Player.

Step 2: Create a New Virtual Machine

1. Click Create a New Virtual Machine.
2. Select Installer disc image file (ISO).
3. Select the Ubuntu ISO file.
4. Click Next.

Step 3: Configure Hardware

1. Enter the VM name and allocate disk space (e.g. 25 GB).
2. Use Customize Hardware to allocate Memory (e.g. 4 GB), Processors (e.g. 2) and Network (NAT).



Step 4: Create the Virtual Machine

1. Review the configuration.
2. Click Finish.
3. The Ubuntu virtual machine will be created.

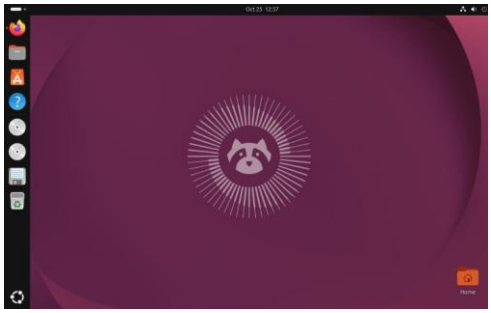
Step 5: Start the Virtual Machine

1. Select the Ubuntu VM.

2. Click Play Virtual Machine.
3. Ubuntu will start from the ISO file.

Step 6: Install Ubuntu

1. Select Install Ubuntu.
2. Follow the installation instructions.
3. Create a username and password.
4. Complete the installation.
5. Restart the virtual machine when prompted.



Commands for Verification

Open the Ubuntu Terminal and execute:

1. `lscpu` – displays the CPU information available to the virtual machine.
2. `free -h` – displays the total, used and available RAM (total, used, free; `-h` shows values in GiB/MiB).
3. `uname -a` – displays information about the Linux kernel and system.
4. `df -h` – displays the available and used disk space.
5. `ip addr` – displays the network interfaces and IP addresses of the virtual machine.

```
ritik_06@ritik_06-VMware-Virtual-Platform:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           671M  2.5M  668M   1% /run
/dev/sda2        20G   11G  7.9G  58% /
tmpfs           1.7G     0  1.7G   0% /dev/shm
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs           1.7G  8.0K  1.7G   1% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
tmpfs           336M   76K  335M   1% /run/user/1000
/dev/sr0        110M  110M     0 100% /run/media/tanishq/CDROM
/dev/sr1        6.1G  6.1G     0 100% /run/media/tanishq/Ubuntu 26.04.1 LTS amd64
ritik_06@ritik_06-VMware-Virtual-Platform:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
    link/ether 00:0c:29:42:79:4d brd ff:ff:ff:ff:ff:ff
    inet 192.168.246.128/24 brd 192.168.246.255 scope global dynamic noprefixroute
       valid_lft 1138sec preferred_lft 1138sec
    inet6 fe80::20c:29ff:fe42:794d/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
ritik_06@ritik_06-VMware-Virtual-Platform:~$
```

Observation: The Ubuntu operating system runs inside VMware as a virtual machine. The VM receives virtual CPU, RAM, storage and network resources from the host computer.

Result: The Ubuntu operating system was successfully implemented as a virtual machine using VMware Workstation, demonstrating operating system/server virtualization.